Linux for Cybersecurity

A practical command-line reference

Personal study and revision reference

Contents

Linux Command Reference 3
Help and Terminal Workflow 5
Files and Directories 6
Users and Permissions 7
Processes and Services 9
Package Management 10
Networking Commands 12

Linux Command Reference

Use commands with the permissions needed for the task only. Arguments beginning with `-` are options; quoting protects paths containing spaces from shell word splitting.

Inspect the current location

Command	Purpose	Safe example
`pwd`	Print the current working directory.	`pwd`
`whoami`	Print the effective user name.	`whoami`
`id`	Show the effective identity and group memberships.	`id`
`date -Is`	Show the current time in ISO-like format.	`date -Is`

These commands inspect state and do not change files. Output depends on the host and account.

List and search

Command	Purpose and arguments	Safe example
`ls -la PATH`	List entries; `-l` uses a detailed listing and `-a` includes dotfiles.	`ls -la "$HOME"`
`find PATH -type f -name PATTERN`	Search recursively; `-type f` selects files and `-name` matches a name pattern.	`find "$HOME/Documents" -type f -name '*.log'`
`grep -n 'TEXT' FILE`	Search file text; `-n` includes line numbers.	`grep -n 'failed' auth.log`
`wc -l FILE`	Count newline-delimited lines.	`wc -l auth.log`

Searching a broad tree can take time and may produce permission errors. Search only paths you are authorized to inspect.

Read files

Command	Purpose	Safe example
`cat FILE`	Write the file contents to standard output.	`cat notes.txt`
`less FILE`	Read a file interactively; press `q` to quit.	`less /var/log/syslog`
`head -n N FILE`	Show the first N lines.	`head -n 20 auth.log`
`tail -n N FILE`	Show the last N lines.	`tail -n 20 auth.log`

Command	Purpose	Safe example
`tail -f FILE`	Follow appended data until interrupted with Ctrl+C.	`tail -f app.log`

Log locations and access permissions vary by distribution. Do not copy sensitive log data into an unapproved location.

Pipes and redirection

Syntax	Effect	Example		
`A \	B`	Send A's standard output to B's standard input.	`grep 'failed' auth.log \	wc -l`
`A > FILE`	Replace FILE with A's standard output.	`printf '%s\n' 'review' > review.txt`		
`A >> FILE`	Append standard output to FILE.	`date -Is >> review.txt`		
`A 2> FILE`	Redirect standard error.	`find /var/log -type f 2> find-errors.txt`		
`A && B`	Run B only if A exits successfully.	`mkdir review && cd review`		

> overwrites its destination. Verify the path before using redirection. A pipeline generally passes output, not the first command's success status, unless shell options are configured.

Change or create files

Commands that modify the filesystem are marked **Modify**. Check paths before acting.

Command	Purpose and arguments	Safe example
`mkdir -p PATH`	**Modify:** create directories; `-p` also creates missing parents and does not fail if the directory exists.	`mkdir -p "$HOME/lab-notes"`
`touch FILE`	**Modify:** create an empty file or update timestamps.	`touch "$HOME/lab-notes/todo.txt"`
`cp SOURCE DEST`	**Modify:** copy a file.	`cp notes.txt notes.backup.txt`
`mv SOURCE DEST`	**Modify:** move or rename; an existing destination may be replaced depending on options and prompting configuration.	`mv draft.txt draft-reviewed.txt`
`rm FILE`	**Destructive:** remove a directory entry; there is usually no simple undo.	Avoid in routine study examples; confirm exact target and backups first.

Avoid recursive deletion options during exploration. Shell wildcards can expand to more files than intended. Inspect the exact target with `printf '%s\n' PATH` or `ls` before any removal.

Revision points

- `pwd`, `ls`, and `cd` are the core navigation commands highlighted in the accessible episode 1 chapters.
- Inspect before changing; quote variable paths; check permissions and scope.
- `>` overwrites, `>>` appends, and pipes connect command output.
- Treat `rm`, recursive options, and privileged commands as high-impact operations.

Help and Terminal Workflow

The playlist feed includes an episode titled "HELP!! (for when you suck at Linux)" and a terminal-shortcuts video. This page collects safe ways to find command help and work efficiently in a shell; exact lesson demonstrations were not available for verification.

Find command documentation

Command	Purpose	Example
`man COMMAND`	Open the installed manual page; press `q` to quit.	`man ls`
`COMMAND --help`	Show command-specific usage when supported.	`cp --help`
`apropos KEYWORD`	Search manual-page names and short descriptions.	`apropos permissions`
`type COMMAND`	Show how the shell resolves a command name.	`type cd`
`command -v COMMAND`	Print the executable path or shell resolution where available.	`command -v python3`

Manual-page sections can contain the same name for different interfaces; use `man SECTION NAME` when the target is ambiguous. Examples include `man 1 printf` for the shell command and `man 3 printf` for a C library function.

Shell history and editing

Action	Effect	Caution
Up / Down arrow	Recall earlier or later commands in the current shell history.	Review a recalled command before pressing Enter.
Tab	Complete a command or path when the shell can identify a match.	Multiple matches may require another Tab or more characters.
Ctrl+C	Send an interrupt to the foreground process.	May interrupt a command that is changing data; understand what is running.

Action	Effect	Caution
Ctrl+L	Clear the visible terminal area in common shells.	This clears the display, not shell history or logs.
`history`	Show commands recorded in the shell history.	History can contain sensitive values; avoid putting secrets directly in commands.

Safer command habits

- Read the command and arguments before pressing Enter, especially after using history completion.
- Use `man` or `--help` to check flags before combining options.
- Quote paths and variables that may contain spaces or shell metacharacters.
- Avoid placing passwords, tokens, or other secrets in command arguments; shells may retain them in history or expose them to process listings.
- Use `clear` or Ctrl+L only to manage the display; do not mistake it for secure deletion.

Revision points

- Prefer the local manual page for distribution-specific details.
- Check a command's target and flags before running it with elevated privileges.
- Shell history and process listings can expose sensitive command-line values.

Files and Directories

Navigation model

Linux uses a single directory tree rooted at `/`. A path beginning with `/` is absolute; a relative path is interpreted from the current directory. `.` means the current directory, `..` its parent, and `~` commonly expands to the current user's home directory.

Command	Behavior	Safe example
`pwd`	Inspect current directory.	`pwd`
`ls -la PATH`	Inspect contents, including hidden entries.	`ls -la /etc`
`cd PATH`	Change the current shell directory.	`cd "$HOME/Documents"`
`realpath PATH`	Print a resolved absolute path when available.	`realpath ./notes.txt`

File operations

Command	Effect	Example and caution
`mkdir -p DIR`	**Modify:** create a directory and missing parents.	`mkdir -p "$HOME/labs/week-1"`

Command	Effect	Example and caution
`touch FILE`	**Modify:** create an empty file or update timestamps.	`touch "$HOME/labs/todo.txt"`
`cp SRC DEST`	**Modify:** copy a file.	`cp config.txt config.txt.bak`; check whether DEST already exists.
`mv SRC DEST`	**Modify:** move or rename.	`mv draft.md final.md`; verify both paths first.
`rm FILE`	**Destructive:** remove a file entry.	Do not use until the exact path and recovery needs are checked.

For file metadata, `stat FILE` shows details such as size and timestamps. `file FILE` reports the type inferred from content. These inspect a file; they do not prove that it is safe to open or execute.

Reading and locating content

Use `less FILE` for interactive reading, `head -n 20 FILE` for the first lines, and `tail -n 20 FILE` for the last lines. Search a known text file with `grep -n 'pattern' FILE`. Locate files by name with `find DIR -type f -name '*.conf'`.

Avoid searching the entire filesystem without a reason. Permission restrictions can generate errors, and broad searches may be slow or expose data outside the intended scope.

Common mistakes

- Running a relative path from the wrong working directory.
- Forgetting that dotfiles are hidden from a plain `ls` listing.
- Assuming `mv` or `cp` will always ask before replacing a destination.
- Using wildcards with modifying commands before checking what they match.

Revision points

- Confirm the working directory and target path before modifying files.
- Use inspection commands first and quote paths containing whitespace.
- Treat deletion and overwrite behavior as irreversible until recovery is tested.

Users and Permissions

Identities and groups

Processes run with user and group identities. Group membership can grant access to shared files or devices. Use `whoami` to inspect the effective user, `id` to inspect numeric and named identity/group information, and `groups` to list group memberships.

Permission model

For each filesystem object, traditional Unix permissions define read (`r`), write (`w`), and execute (`x`) access for the owner, group, and others.

Object	Read	Write	Execute
Regular file	Read contents	Change contents	Run as a program/script if other requirements are met
Directory	List names	Create, remove, or rename entries (subject to other controls)	Traverse/search the directory

The meaning of directory permissions differs from file permissions. A directory may need execute permission to traverse it even when its names are visible.

Inspect and change permissions

Command	Purpose	Example
`ls -l FILE`	Inspect owner, group, and permission bits.	`ls -l notes.txt`
`stat FILE`	Inspect detailed metadata.	`stat notes.txt`
`chmod MODE FILE`	**Modify:** change permission bits.	`chmod 640 report.txt` grants owner read/write, group read, and others none.
`chown USER:GROUP FILE`	**Elevated modify:** change ownership; often requires administrator rights.	Use only under an approved change plan.
`sudo COMMAND`	Request execution with elevated privileges under local policy.	`sudo systemctl status ssh` (read-only status operation).

Numeric `chmod` values add permission bits: read=4, write=2, execute=1. `640` therefore means owner `6` (read+write), group `4` (read), others `0` (none). Confirm the intended access before changing permissions.

Least privilege

Use the narrowest account and permission set that supports the task. Avoid making files globally writable or running an interactive session as root. Elevated access increases the impact of mistakes and should be limited, auditable, and authorized.

Common mistakes

- Applying `chmod 777` as a quick fix, granting everyone read, write, and execute access.
- Confusing file write permission with permission to remove its directory entry.
- Assuming `sudo` makes a command safe or reversible.
- Changing ownership recursively without a verified target and change plan.

Revision points

- Inspect user, groups, owner, and mode before diagnosing access.
- Understand `rwx` separately for owner, group, and others.
- Make the smallest permission change needed; use elevation only when required.

Processes and Services

Processes

A process is a running instance of a program. Process identifiers (PIDs) identify processes on a host. Use inspection commands to understand system state before attempting to change it.

Command	Purpose	Example
`ps -ef`	Inspect a process listing; `-e` selects all processes and `-f` uses a full-format listing.	`ps -ef`
`ps -p PID -o pid,ppid,user,stat,cmd`	Inspect selected fields for one PID.	`ps -p 1234 -o pid,ppid,user,stat,cmd`
`top`	Interactively inspect processes and resource use; press `q` to quit.	`top`
`pgrep -a NAME`	Find process IDs and command lines matching a name.	`pgrep -a ssh`

PIDs change over time and may be reused. Verify the target and owner before taking action.

Signals

`kill -TERM PID` requests graceful termination using the TERM signal. It modifies system state and can interrupt work. `kill -KILL PID` forces termination and prevents cleanup; avoid it unless a documented recovery procedure requires it. Never infer that a process is malicious from its name alone.

Services with systemd

Command	Effect	Example
`systemctl status SERVICE`	Inspect service state and recent status details.	`systemctl status ssh`
`systemctl list-units --type=service`	Inspect loaded service units.	`systemctl list-units --type=service`
`sudo systemctl restart SERVICE`	**Elevated modify:** restart a service and may interrupt users.	Use only after impact and maintenance requirements are understood.
`sudo systemctl enable SERVICE`	**Elevated modify:** configure a service to start at boot.	Make only as an approved configuration change.

Service names and service managers differ by distribution. A service being active does not establish that its configuration is secure.

Common mistakes

- Killing a process before identifying what it does and who depends on it.
- Using force termination before attempting an orderly stop.
- Restarting a production service without checking impact and change procedures.
- Treating service status as proof that an application is healthy or secure.

Revision points

- Inspect PID, owner, command, and service context before intervention.
- Prefer a graceful, documented action over a forced one.
- Separate "running" from "correctly configured" and "healthy."

Package Management

The accessible playlist feed includes an episode titled "apt, dpkg, git, Python PiP (Linux Package Management)." Package commands differ across distributions. The examples below use Debian-family `apt` and `dpkg`; use the package manager documented for the host.

Inspect installed and available packages

Command	Effect	Safe example
`apt show PACKAGE`	Inspect package metadata available to configured sources.	`apt show curl`
`dpkg -l PATTERN`	Inspect locally known package status; `-l` requests a listing.	`dpkg -l 'openssh*'`
`dpkg -S PATH`	Find which installed package owns a path.	`dpkg -S /usr/bin/ssh`
`apt list --installed`	List installed packages.	Output may be lengthy; pipe to a search only when useful.

Update and install

Command	Effect	Precautions
`sudo apt update`	**Elevated modify:** refresh package indexes from configured repositories; does not itself upgrade installed packages.	Verify repository configuration and use an authorized host.
`sudo apt install PACKAGE`	**Elevated modify:** install a package and required dependencies.	Review the proposed package changes before confirming.
`sudo apt upgrade`	**Elevated modify:** upgrade installed packages according to local APT behavior.	Can change services or dependencies; follow change and maintenance procedures.

Command	Effect	Precautions
`sudo dpkg -i FILE.deb`	**Elevated modify:** install a local Debian package file.	Verify provenance and integrity; handle dependencies through the package manager.

Package operations access configured repositories and change the host. Do not install tools on systems without authorization. Avoid piping downloaded scripts directly into a privileged shell.

Python packages

Use a virtual environment for project-specific Python dependencies. This limits package changes to the project environment and helps avoid conflicts with operating-system-managed Python packages.

```
python3 -m venv .venv
. .venv/bin/activate
python -m pip install PACKAGE
python -m pip show PACKAGE
```

The first three commands create and activate a project environment and install a named package; `pip show` inspects package metadata. Package names and dependencies should be reviewed before installation. Do not use `sudo pip` as a routine workaround for permission errors.

Git basics

Git is a version-control tool, not a system package manager. The following commands inspect local repository state:

Command	Purpose
`git --version`	Show the installed Git version.
`git status`	Inspect the current repository's changes and branch state.
`git log --oneline -5`	Inspect the latest five commits in compact form.

Cloning or fetching contacts a remote and writes repository data locally. Confirm the remote source and destination path before using network operations.

Revision points

- `apt` manages repository packages on Debian-family distributions; `dpkg` works with local package database and package files.
- Refreshing package indexes is different from upgrading installed software.
- Package installation and upgrades change the host and usually require elevation.
- Keep Python project dependencies in a virtual environment; use Git to track project changes.

Networking Commands

Use read-only network commands first. Results depend on permissions, installed tools, host configuration, DNS, and network policy.

Command	What it inspects	Example and notes
`ip address show`	Local interfaces and assigned addresses.	`ip address show`
`ip route show`	Local routing table.	`ip route show`
`ss -tulpen`	Listening and established sockets; options select TCP/UDP, listening sockets, numeric endpoints, processes, and extended details.	`ss -tulpen`; process details may require elevated access.
`ping -c 4 HOST`	Attempts four ICMP echo requests.	`ping -c 4 example.com`; filtering or policy may prevent replies.
`getent hosts HOST`	Queries configured name-service sources for host records.	`getent hosts example.com`
`curl -I URL`	Requests response headers from a URL.	`curl -I https://example.com`; contacts the remote host and may be logged.
`traceroute HOST`	Attempts to show network hops to a host.	`traceroute example.com`; availability and results depend on the path and installed package.

Troubleshooting sequence

1. Inspect interface state and address configuration with `ip address show`.
2. Inspect the route table with `ip route show`.
3. Check name resolution with `getent hosts HOST`.
4. Test reachability only when appropriate; a missing ping reply does not prove a host is offline.
5. Inspect local listening sockets with `ss` and compare them with expected services.

Common mistakes and precautions

- Treating ICMP failure as proof that an application is unreachable.
- Running active probes against systems without authorization.
- Assuming a local listening socket is exposed externally; firewalls and routing affect reachability.
- Sharing outputs that contain internal addresses, usernames, or process details without review.
- Using `curl` or other clients to send requests to systems outside the approved scope.

Revision points

- Establish local interface, route, and DNS state before concluding where a failure occurs.
- Distinguish local listening state from remote reachability.
- Stay within authorized systems and record the exact host and time when troubleshooting.

Sources and access notes

NetworkChuck Linux for Hackers playlist: playlist. Episode 1 with chapter markers: video. Command references: Coreutils index, ip manual, ss manual, and curl manual.

The public feed exposed episode titles but not transcripts. The notes follow the episode topic sequence and describe command behavior from standard documentation; they do not attribute commands to unseen lesson details.